

investigation_20260925

BOB-172 — rutracker SEARCH endpoint HTTP 403: fresh re- measurement + header-tuning test + roster-mirror check

Field	Value
Revision	1
Last modified	2026-09-25T10:30:00Z
Item	BOB-172 (Bug, Ready for testing — left open, NOT closed by this investigation)
Method	superpowers:systematic- debugging (root cause before fix; §11.4.199 exact-reproduction- sequence)
Secrets	No credential or cookie VALUE appears below (§11.4.10). Every probe below is unauthenticated — no .env read, no cookie jar used.

1. Verdict

Outcome 2 — structurally blocked. The rutracker search endpoint's 403 is a genuine Cloudflare edge-level bot-mitigation ("Managed Challenge" class: cf-mitigated: challenge response header), re-confirmed live today with the exact BOB-172 signature, and — newly tested this session — **identical whether the request carries no**

special headers or a full realistic Chrome-browser header set. This rules out client-header tuning as a fix. All three configured roster mirrors (rutracker.org, rutracker.net, rutracker.nl) were also probed live; none offers an escape (two are Cloudflare-gated identically, the third has a broken TLS chain from this host). No code change in this repository can pass this challenge without external infrastructure (a JS-capable challenge solver) or an operator-supplied `cf_clearance` cookie — both of which are §11.4.122 operator-owned decisions, exactly the precedent BOB-235 already established for kinozal’s Cloudflare wall.

Separately — and this matters for scoping the remaining work — **acceptance criteria (b) and (c) of BOB-172’s own text (loud error reporting instead of a silent “empty”) are already implemented and tested**, via `_classify_upstream_http_status / _check_search_response`, landed under this same item plus its two follow-on gap-closure items (BOB-177, BOB-179). I ran that test suite live this session (51 tests, all green — §2 below). What remains open is **acceptance (a)** (which this investigation answers: permanent Cloudflare policy, not rate/reputation/header-fixable) and **acceptance (d)** (stop advertising rutracker as a live search source in the README/docs — NOT yet done for the *private*-tracker fan-out; only unrelated *public*-tracker entries in docs/MERGE_SEARCH_DIAGNOSTICS.md’s DEAD_PUBLIC_TRACKERS list carry this treatment).

2. Existing-fix verification (source-side, already landed under this item + its follow-ons)

Before re-measuring the live wall, I confirmed the source-side portion of BOB-172’s own acceptance criteria is in place and passing, so this investigation’s job is squarely the remaining external-capability question, not a re-implementation:

```
$ .venv/bin/python -m pytest tests/unit/merge_service/
test_bob172_tracker_http_error_not_empty.py \
    tests/unit/merge_service/test_bob177_guard_wiring_collapse.py \
    tests/unit/merge_service/test_bob179_stated_gaps_closure.py -q
--import-mode=importlib
.....
[100%]
51 passed in 3.49s
```

download-proxy/src/merge_service/search.py's
_classify_upstream_http_status() (module-level, lines ~376-445)
explicitly cites BOB-172's 2026-08-21 measurement in its own docstring,
classifies any non-2xx as a diagnostic (upstream_http_403, etc.) rather
than letting the refusal body fall through to the row parser and render
as a false "empty", and appends the exact BOB-172 language ("bot-
protection refusal ... supplying credentials or refreshing cookies does
not address it") when challenge markers are present in the body.
_check_search_response() wires this into both of _search_rutracker's
auth paths (cookie path and username/password path). This is what I
re-verified is live and green — **I made no changes to this code**, since
no bug was found in it.

3. Fresh live re-measurement (2026-09-25, THIS session, unauthenticated)

Reproduced with the exact search-endpoint request shape the item's
own evidence uses (§11.4.199 — same path, same query param shape
as the code's `f"{base_url}/forum/tracker.php?{urlencode({'nm':
query, 'fo': 1})}"`), from this host, no cookies, no credentials:

```
$ curl -sS -D headers_index_default.txt -o body_index_default.html  
-w "HTTP:%{http_code} SIZE:%{size_download} TIME:%{time_total}\n"  
\n    --max-time 15 "https://rutracker.org/forum/index.php"  
HTTP:200 SIZE:96460 TIME:0.093309
```

```
$ curl -sS -D headers_search_default.txt -o  
body_search_default.html -w "HTTP:%{http_code} SIZE:%  
{size_download} TIME:%{time_total}\n" \  
    --max-time 15 "https://rutracker.org/forum/tracker.php?  
nm=debian&fo=1"  
HTTP:403 SIZE:5381 TIME:0.027218
```

Marker census on the 403 body (grep -a used defensively after a
control-needle catch below — §11.4.273):

```
challenge markers: just a moment=1 cf-chl=0 challenge-  
platform=1 cf-browser-verification=0  
login markers:     login_username=0 login_password=0  
bb_session=0
```

Response headers on the 403 carry the authoritative, non-body-
dependent Cloudflare signal:

```
server: cloudflare  
cf-mitigated: challenge
```

```
content-security-policy: ... script-src 'nonce-...' 'unsafe-eval'  
https://challenges.cloudflare.com ...
```

This is the exact same shape as the item's own 2026-08-21 measurement (403, challenge markers present, zero login markers, sub-100ms — too fast for a real remote search), independently reproduced 35 days later. The finding has not gone stale.

Control-needle note (§11.4.273, recorded honestly because it is instructive): my first pass at `grep`-checking the 200 `index.php` body for a sanity string (RuTracker) silently returned zero matches with plain `grep -c -i`, because that body is served `charset=Windows-1251` and the shell's `LANG=en_US.UTF-8` made `grep` treat the non-UTF-8 bytes as unreadable and refuse to match — a textbook false-null, not evidence the string was absent (`file` confirmed “Non-ISO extended-ASCII text”; the literal bytes for RuTracker.org are visible in a hex dump at offset 0x90). Re-run with `grep -a` (treat as text) found 171 matches. The 403 body itself is `charset=UTF-8` (confirmed by both its header and `file`), so the marker counts quoted above were never affected by this — I re-ran them with `-a` regardless, defensively, with identical results, and record the near-miss per the same discipline BOB-172's own provenance section used for its Gap-A correction.

4. NEW this session — client-signature (header) test, per the task's outcome-1 requirement

The task asked me to test, not assume, whether the merge service's *own* request signature (headers/timing/UA) — as opposed to `rutracker.org`'s blanket policy — is what triggers the challenge, since `_tracker_session_kwargs()` sends only `{"trust_env": True}` and no browser-shaped headers at all (aiohttp's bare default `User-Agent: Python/x.y aiohttp/x.y.z`).

I sent the identical search request with a full, realistic Chrome-on-Linux header set (`User-Agent`, `Accept`, `Accept-Language`, `Accept-Encoding`, `Referer`, `sec-ch-ua*`, `Sec-Fetch-*`, `Upgrade-Insecure-Requests`), decompressing the response properly (`--compressed`; the raw first attempt without it produced a Brotli-compressed body that a naive `grep` again silently failed to match — the same control-needle class of near-miss caught and corrected before I drew a conclusion from it):

```
$ curl -sS --compressed -D headers_search_browserlike2.txt -o  
body_search_browserlike2.html \
```

```

-w "HTTP:%{http_code} SIZE:%{size_download} TIME:%{time_total}
\n" --max-time 15 \
-H "User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/
537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36" \
-H "Accept: text/html,application/xhtml+xml,application/
xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8" \
-H "Accept-Language: en-US,en;q=0.9" \
-H "Referer: https://rutracker.org/forum/index.php" \
-H 'sec-ch-ua: "Chromium";v="128", "Not;A=Brand";v="24",
"Google Chrome";v="128"' \
-H "sec-ch-ua-mobile: ?0" -H 'sec-ch-ua-platform: "Linux"' \
-H "Sec-Fetch-Dest: document" -H "Sec-Fetch-Mode: navigate" -H
"Sec-Fetch-Site: same-origin" -H "Sec-Fetch-User: ?1" \
-H "Upgrade-Insecure-Requests: 1" \
"https://rutracker.org/forum/tracker.php?nm=debian&fo=1"
HTTP:403 SIZE:3597 TIME:0.021067

```

Same verdict, same authoritative header:

```

cf-mitigated: challenge      (identical to the bare-headers request
- headers_search_default.txt has the same line)
challenge markers: just a moment=1 challenge-platform=1
login markers:      login_username=0 login_password=0

```

Finding: headers make no difference. A full browser-shaped request set and a bare curl request set both received the identical `cf-mitigated: challenge` edge decision, in ~21-27ms — far too fast for any client-side JS challenge to have been attempted, meaning Cloudflare decided at the edge, before serving any page, using signals a static HTTP client (any HTTP library — `aiohttp`, `curl`, `requests`) structurally cannot present regardless of which headers it sends: most likely TLS/JA3-JA4 fingerprint and/or the absence of a prior browser-issued challenge-clearance cookie. Setting a convincing `User-Agent` string does not touch either of those signals. This matches the mechanism already documented for the unrelated kickass public tracker at `docs/MERGE_SEARCH_DIAGNOSTICS.md:150-153` (“`cf-mitigated: challenge` response header, identical whether curl sent a spoofed browser `User-Agent` or its own default ... same unfixable-from-our-side class as `rutracker/BOB-172`”).

Control probes on the same host, to characterize scope (is it only the search path, or the whole forum):

```

GET /forum/index.php      -> 200  (no cf-mitigated header at all)
GET /forum/tracker.php    -> 403  cf-mitigated: challenge  <- the
search endpoint the code uses
GET /forum/search.php     -> 403
GET /forum/viewforum.php  -> 403

```

The landing page is open; everything past it (search, browse) is gated by the same site-wide Cloudflare policy. This is consistent with a deliberate anti-scraping posture on rutracker's part, not a signature the merge service happens to trip.

5. NEW this session — roster-mirror check

trackers.py:102 declares three configured mirrors for rutracker: rutracker.org, rutracker.net, rutracker.nl (the code's `base_url = os.getenv("RUTRACKER_MIRRORS", "https://rutracker.org").split(",")[0].strip()` only ever tries the FIRST one unless the operator sets `RUTRACKER_MIRRORS` explicitly). I probed the search path on all three live, unauthenticated:

```
rutracker.org  index=200          search=403  cf-mitigated:
challenge
rutracker.net  index=502 (upstream) search=403  cf-mitigated:
challenge     (same wall, plus its own outage)
rutracker.nl   index=curl:60 (TLS: unable to get local issuer
certificate)   search=same TLS failure
```

rutracker.nl's certificate is issued by Let's Encrypt for the correct CN (CN=rutracker.nl), so this reads as a broken/incomplete certificate chain from this host rather than an obvious MITM — but it is unusable as measured regardless of cause, including with `-k` (which itself failed with an HTTP/2 protocol error). **No configured mirror offers an escape from the Cloudflare wall**; a mirror switch is not a viable fix path.

6. Answer to acceptance criterion (a)

Determine whether the 403 is permanent policy, rate/reputation-based, or triggered by a client signature the plugin can legitimately present — by measurement, not assumption, and WITHOUT evasion techniques that would violate the site's terms.

Measured answer: this is Cloudflare edge-level bot-mitigation policy (Managed Challenge class), not a client-signature issue we can legitimately fix, and not distinguishable from rate/reputation-based blocking by the tests available to an unauthenticated, non-JS-executing client — the sub-30ms response time on every probe (bare headers, full browser headers, alternate mirrors) rules out anything that would require Cloudflare to inspect request history or apply escalating suspicion; it looks like the endpoint is challenge-gated

unconditionally for any client lacking a prior browser-issued clearance token. I did **not** attempt any evasion technique (no TLS-fingerprint spoofing library, no headless-browser JS-challenge solver, no scraping proxy) — every probe above is a plain, honestly-identified HTTP client presenting real or realistic headers, which is exactly what the item's own acceptance criterion asked for and what “without evasion techniques” rules out going further than.

7. Recommended disposition (mirrors the BOB-235 format; I am NOT applying this myself — no workable-items CLI access, and the task explicitly asked me not to)

Recommend: mark BOB-172 Operator-blocked, citing this investigation and docs/qa/BOB-172/rutracker_search_403_20260821.log + fix_evidence_20260822.log, with the following enumerated operator choices (BOB-235's own four-option shape, adapted to rutracker's specifics):

- 1. Accept a challenge-solving service.** Deploy FlareSolvrr (or equivalent headless-browser Cloudflare-challenge solver) as a sidecar, and have `_search_rutracker` route its search GET through it instead of a bare `aihttp` request. Not deployed today (same gap noted for kinozal in BOB-235). Infrastructure + maintenance cost; introduces a JS-execution dependency into the merge path.
- 2. Accept a supplied `cf_clearance` cookie.** `RUTRACKER_COOKIES` already exists as an env var and its cookie-path branch in `_search_rutracker` (lines 1607-1680) already sends whatever cookie dict is supplied — but it only checks for `bb_session` (the forum login cookie) and never checks for, or requires, a `cf_clearance` cookie, which is the actual token Cloudflare's edge is checking for on this path. A `cf_clearance` value is short-lived (bound to the issuing IP + user-agent) and would need an operator-run refresh cadence (mirroring the existing `scripts/load-tracker-cookies.sh` autoload pattern this project already uses for tracker session cookies).
- 3. Route rutracker searches through Jaxx's rutracker indexer instead of the direct HTTP path**, if Jaxx's own scraper has a working Cloudflare-bypass for this tracker (unverified in this session — out of scope; a follow-up investigation would need to check `boba-jaxx's` indexer health for rutracker specifically).
- 4. Mark rutracker search unsupported per §11.4.90 / BOB-172 acceptance (d)** — stop presenting it as a live search source: qualify

or remove the unconditional “RuTracker” mention in README.md:167 (currently listed under credentials with no caveat) and add the same honest-capability-boundary note docs/MERGE_SEARCH_DIAGNOSTICS.md already carries for the unrelated kickass/eztv/etc. public-tracker entries, cross-referencing this item. (The download-proxy / credential-registration flow for rutracker downloads is unaffected — this only concerns the merge-search fan-out.)

Whichever option the operator chooses, the code-side reporting is already correct as of this session (§2): a 403 from rutracker’s search endpoint surfaces to the merge result as `error_type=upstream_http_403` with the bot-protection explanation appended, not as a silent empty — so no user is currently being misled about *why* rutracker contributes nothing; the remaining question is purely whether/how to make it contribute something again, which is the operator’s call.

8. Honest boundary

Measured from **one host, unauthenticated, today (2026-09-25)** — same caveat the original 2026-08-21 measurement carried. I did not test from a different network/IP (rate/reputation-based blocking, if any, could in principle differ by source IP), and I did not test with a genuine browser-issued `cf_clearance` cookie (option 2 above remains untested, not ruled out as a fix — only the *header-tuning-without-a-real-clearance-cookie* path is ruled out by this session’s evidence). The finding is that the current unauthenticated, non-JS-executing code path gets a 403 regardless of header signature; it is not a claim that no legitimate client configuration could ever pass.